Exact Filtered Vector Search Across Execution Environments: Eligibility, Small Collections, and Completed-Call Cost

Akshat Singh Kushwaha
akshatsingh14372@outlook.com
https://github.com/a3ro-dev/qenlo

September 2026

Abstract

Filtered vector search chooses an execution route after a predicate restricts the candidates. We study that decision in Qenlo, an embedded vector store with exhaustive CPU, portable WGPU, and optional tensor execution. The evidence spans historical Windows crossover and eligibility-preparation experiments, Android and Intel Arc device records, A6000 research prototypes, external engines, and a new 1K–100K collection campaign. Cohorts remain separate by source, device, dataset, predicate, and timing boundary. On a historical RTX 4050, CPU and compact-row WGPU exchange the lead between 2,000 and 3,000 eligible 384-dimensional vectors; a static 4,096-row rule incurs up to 23.9% descriptive P95 regret. A separate RTX 4090 ablation reduces sparse-cell P95 by 10.5–32.3% with one-pass eligibility preparation. In the corrected experimental archive, RTX 4090 WGPU completes 100K-by-768 search in 0.897 ms P95 at batch one and 0.896 ms at batch eight with 10% eligibility; tensor CUDA takes 0.671 and 0.384 ms. All twelve corrected engine rows have FP64-oracle recall 1.0. These timings use a selector candidate subsequently rejected after five wins and seven losses in twelve frozen-baseline pairs; they are not measurements of the final local selector. Tested warm post-mutation searches avoid resident-state rebuilds, whereas reopen incurs preparation. ANN wins, correctness failures, unavailable devices, and aggregate-only mobile evidence remain visible. Eligible work, preparation, representation, batching, selection, and device state jointly inform routing; the archive does not establish a transferable threshold or validate an adaptive production router.

1 Introduction and research questions

A filtered query over 100,000 vectors may score only one hundred candidates. The remaining work can be dominated by finding those candidates, dispatching a device, selecting results, or returning them through an application interface. Conversely, an unfiltered batch may amortize dispatch enough to favor exhaustive GPU execution even for a modest collection. Collection size and GPU availability alone provide an incomplete description of the execution problem.

Qenlo supplies a concrete system in which to examine this boundary. Canonical records are shared by rebuildable search structures; changing routes should preserve the logical result contract. The archive contains several generations of implementation and experiment design. Earlier studies emphasize eligible cardinality, predicate representation, and routing regret. The September 5 campaign adds small collections, tensor adapters, resource diagnostics, lifecycle measurements, and a rejected selector. This paper preserves those findings without treating historical or candidate results as measurements of the final source tree.

We ask three questions. Under what measured conditions does the preferred exhaustive route change? How much of a completed call comes from eligibility preparation, selection, synchronization, or derived-state preparation rather than scoring? What deployment conclusions survive differences in numerical accuracy, APIs, persistence, runtime ownership, and evidence quality?

The contribution is a systems study, not a new asymptotic nearest-neighbor algorithm. It combines a localized historical crossover, an eligibility-materialization ablation, a failure-preserving small-collection campaign, and heterogeneous-device observations. A claim ledger records which measurements support each conclusion. A rejected optimization retains scientific value, while its latency does not become a promise about the retained implementation.

2 System model and semantic contracts

Let the live canonical collection at generation g contain normalized FP32 vectors $x_i \in \mathbb{R}^D$, public identifiers i, and metadata m_i. A predicate P defines

$$S_P(g) = \{i : \ i \text{ is live at } g \ \wedge \ P(m_i)\}, \qquad E = |S_P(g)|. \tag{1}$$

For query q, exhaustive search evaluates eligible rows and returns up to k results ordered within each implementation by computed cosine distance and identifier. The ideal cosine distance is

$$d(q, x_i) = 1 - \frac{q^\top x_i}{\|q\|_2 \|x_i\|_2}. \tag{2}$$

A batch contains B queries. Scoring work scales with BED, but that expression omits predicate traversal, memory access, selection, and completed-call overhead.

Exhaustiveness and numerical agreement. We use "exact" in the algorithmic sense of exhaustive candidate coverage. Stored FP32 values, CPU accumulation, GPU arithmetic, and an FP64 reference need not yield identical rankings near ties. Oracle recall is reported independently. Historical dense native rows have recall@10 of 0.99998 despite exhaustive execution; corrected small-campaign rows have recall 1.0. Neither proves universal FP64 equivalence. ANN remains approximate even when observed recall is one. Deterministic engine ordering does not prove common FP64-oracle tie membership.

Canonical and derived state. The Rust store owns vectors, identifiers, metadata, tombstones, and generation. CPU scan layouts, WGPU residency, ANN indexes, and eligibility plans are derived state. Canonical vectors are FP32, normalized with an FP64 norm accumulator at ingestion and query boundaries. Restoration validates stored bytes without renormalizing them. Durable collections add snapshot and write-ahead-log behavior; in-memory benchmark collections have no durable generation. Source inspection supports these mechanisms, not universal power-loss durability or arbitrary concurrent or crash schedules.

3 Execution paths

CPU and portable GPU. Historical CPU artifacts describe FP64 accumulation with AVX2/FMA where available. Audited current CPU code also uses a bounded FP32 candidate stage with FP64 rescoring and reference fallback. Not every CPU arithmetic operation is FP64. These are Qenlo implementations, not bounds on optimized CPU performance. WGPU uses WGSL shaders and native graphics backends. WebGPU and WGSL specify a programming model, not performance portability [14, 15]; positive execution evidence is limited to named devices.

Resident GPU vectors use compact eligible rows, dense masks, or supported shader predicates. Bounded chunks emit candidates for host merging. Under a common total order, an item outside its chunk's top k cannot enter the global top k, since at least k items in that chunk precede it. This standard reduction explains bounded readback; it does not remove floating-point differences.

Eligibility compilation. An `EligibilityPlan` binds a predicate result to generation and records eligible count, representation, row IDs, and preparation diagnostics. One-pass compilation avoids counting eligibility and traversing it again to materialize rows. A bounded cache keyed by generation and predicate reuses plans only while that identity remains valid. Exact row execution validates ordering, bounds, liveness, and predicate membership. The historical ablation holds GPU scoring fixed across two-pass, one-pass, and warmed cached paths; it does not recalibrate the earlier laptop's crossover.

Tensor execution. The optional Python `TorchIndex` owns vector and ID tensors and supports CPU, CUDA, and MPS selection in source; only CPU and CUDA have measurements in this paper. The repository's conformance test is CPU-focused, so source-level device support should not be read as positive evidence for every CUDA or MPS device. A collection-derived snapshot records its generation and rejects staleness at search entry during sequential use. It neither holds a native collection lock throughout computation nor rechecks afterward; concurrent mutation guarantees are unestablished. A detached array index has no durable collection identity.

Both are derived computation, not another durable database. Replays prepare filtered matrices before query timing. Tensor allocation fields exclude allocator and library overhead.

Mutation and route observability. Resident updates append rows or update liveness without rebuilding when capacity and layout permit; rebuild remains the fallback. CPU-only operation avoids GPU initialization. Required-GPU mode returns errors on failure, while automatic mode records fallback. Static routing and hardware-bound profiles exist, but no retained experiment validates a calibrated profile on held-out decisions. File-backed commits validate and stage durability before publication; this statement does not apply to in-memory collections.

4 Methodology and evidence boundaries

A comparison needs source and archive identity, hardware and software, dataset and query partition, $N, D, B, k, E/N$, predicate representation and semantics, API, timing, and correctness gate. Numerical observations stay within named cohorts; cross-cohort conclusions are qualitative synthesis. Historical endpoints with differing revisions remain descriptive context, not controlled ablations.

Timing. Native timing starts at the host search call and ends with host-visible ordered results, including in-call eligibility, transfers, dispatch, synchronization, readback, and merge. Load, construction, warmup, and oracle work are excluded. External small-campaign adapters also return host-visible identifiers and distances but receive a prefiltered canonical matrix. Their build field includes matrix and index construction; the metadata traversal creating eligible IDs precedes that timer. Even `build_and_filter_prepare_ns` omits part of preparation. Chroma includes Python collection bindings and serialization. These are scoped completed calls, not identical database APIs.

CUDA events are separate from host calls. Scoring, selection, and backend diagnostics overlap: summing them or subtracting them from P95 invents an unmeasured phase. Lifecycle records separate mutation, collection opening, and first search. Throughput fields including validation in their wall interval are not inverse query latency and are not used for cross-engine QPS rankings.

Samples and correctness. New synthetic cells generally use three repetitions of 64 evaluation queries: 192 batch-one calls, 24 batch-eight calls, or three batch-64 calls. The real fixture uses 5,000 queries per repetition and 939 batch-16 calls across three runs. Matrix `sample_count` means native batch calls but external replay repetitions; Table 3 restores common calls and runs notation. Per-run nearest-rank percentiles are reduced by the lower median. Replays preserve native query order and truth and validate membership, distances, eligibility, and recall outside timing. Repeated queries are not independent deployment workloads.

Historical Windows and eligibility cells use five runs of 5,000 held-out calls, $k = 10$, $B = 1$, and 200 warmups; the statistic is median run P95. A6000 tables summarize per-call distributions within cells; synthetic A6000 has 500 calls per system and cell. Android and Arc provide aggregate percentiles. No figure claims significance or draws error bars. Lines are guides. Sequential execution, post-hoc choices, and small run counts limit causal and statistical interpretation.

5 Evidence cohorts and source identity

Table 1: Cohort map. Observations describe archive volume, not independent replications. Codes identify source roles below.

Cohort	Hardware / data	Scope	Evidence
Small S0/S1/S2	Ada, A40, 4090; generated, AG News	1K–100K, 128–768D	182 status rows
Localization H1	RTX 4050/DX12; AG News	100K, 384D, $E = 2$K–10K	300,000 raw calls
Endpoints H0	RTX 4050; AG News	100K, sparse/dense	225,000 raw calls
Eligibility H2	RTX 4090/Vulkan; AG News	100K, 384D, three modes	300,000 raw calls
Linux H3	Cloud CPU; AG News	100K, 384D, seven engines	10,500 raw calls
A6000 real H4	CUDA; TopK ModernBERT	1M, 768D, strict predicate	20,000 raw calls
A6000 synthetic H5	CUDA; generated FP32	1M, 768D, four E	4,000 raw calls
Android H6	Mali-G615/Vulkan	10K/100K, 384D	21 aggregate cells
Intel Arc H7	Arc/Vulkan	10K/100K, 384D	21 aggregate cells

S0 is frozen archive `f37e744c`; S1 experimental archive `730c0500`; S2 corrected supplement `bb195fed`. S1/S2 contain the lane-minimum candidate. Local WGSL is byte-identical to S0's simpler selector. Manifests refer to dirty-worktree context around HEAD `b22d3b5`; archive hashes, not that commit alone, identify measured code. Matrix label `current-gpu` is an artifact key, not final local behavior.

H1 records common revision `3e2a4a9`, with dirty worktree; configurations/outputs are part of identity. H0 has several revisions. H2 is archive `26e0ccc0`. H3 is `bb51987`, mirrored and counted once. H4/H5 have different A6000 environments and research CUDA implementations. H6/H7 identify device-lab run IDs rather than reconstructible executable archives.

Windows data derive from DTU AG News embeddings [9]; Linux uses a separately prepared fixture. Equal names do not imply equal bytes or partitions. The small campaign retains its real fixture/checksums. A6000 uses TopK Bench's million-vector 768D ModernBERT corpus [13]; strict/provider-compatible predicates are separate. Appendix B expands provenance and exclusions.

6 Small-collection campaign

6.1 Coverage and common sweep

The matrix has 182 rows: 131 completed, 42 `failed_or_unavailable`, seven failed, and two invalid-harness. Only 130 are qualified. Completed USearch batch-64 remains visible at recall 0.3453125. The manifest has fifteen configurations: ten primary offers plus five reference/deep/retry/fix follow-ups, neither fifteen device replications nor ten successful devices. Available RTX 2000 Ada, two A40 configurations, and reference RTX 4090 produced results; unavailable L4, RTX 3090, A6000, and community offers remain visible.

Table 2: Common synthetic workloads, $k = 10$. Six selected cells, not a factorial sweep.

Cell	N	D	B	E/N
C1	1,000	128	1	100%
C2	1,000	384	16	1%
C3	10,000	128	16	100%
C4	10,000	384	1	1%
C5	100,000	128	1	1%
C6	100,000	384	16	100%

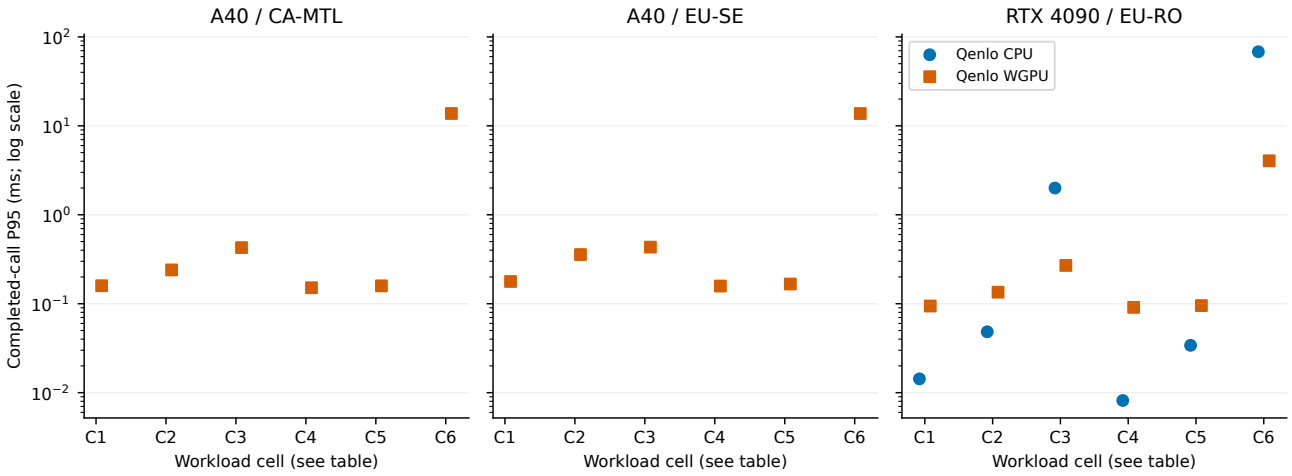

Figure 1: S1 `730c0500`, synthetic common cells, native completed-call P95. Panels are separate hosts. WGPU uses the candidate; no interpolation/error bars. Current-CPU rows are absent on A40, so those panels do not establish CPU/GPU pairs. All displayed FP64 recalls are 1.0. Vertical axis is logarithmic.

On reference RTX 4090, CPU wins C1, C2, C4, C5; WGPU wins dense batched C3/C6. At C5, 100K total rows leave 1K eligible: CPU takes 0.0341 ms P95 versus WGPU's 0.0953 ms. At C6, WGPU takes 4.0479 ms versus CPU's 68.0575 ms. Dimension, batching, and eligibility differ; these are route observations, not isolated size effects. A40 configurations must not supply a pooled device curve.

6.2 Corrected 100K-by-768 supplement

S2 ran on Linux, RTX 4090/Vulkan, driver 580.159.04, AMD Ryzen Threadripper 7960X. Generated data record CRC32 `50b69280`. Table 3 verifies all twelve rows against raw records/matrix. They used the candidate subsequently rejected by local source.

Table 3: S2, 100K×768, completed-call ms. FP64 recall 1.0 throughout. Calls/runs counts batch calls across runs; replays follow native batches. External matrices are prefiltered outside timing.

Workload	Engine	P50	P95	Recall	Calls/runs
Full, B=1, k=1	Qenlo CPU	9.010	9.409	1.0	192 / 3
Full, B=1, k=1	Qenlo WGPU	0.870	0.897	1.0	192 / 3
Full, B=1, k=1	FAISS Flat	14.483	14.800	1.0	192 / 3
Full, B=1, k=1	NumPy	14.507	15.343	1.0	192 / 3
Full, B=1, k=1	Torch CPU	18.324	18.695	1.0	192 / 3
Full, B=1, k=1	Torch CUDA	0.654	0.671	1.0	192 / 3
10%, B=8, k=64	Qenlo CPU	27.402	28.599	1.0	24 / 3
10%, B=8, k=64	Qenlo WGPU	0.842	0.896	1.0	24 / 3
10%, B=8, k=64	FAISS Flat	2.716	3.055	1.0	24 / 3
10%, B=8, k=64	NumPy	5.972	6.202	1.0	24 / 3
10%, B=8, k=64	Torch CPU	2.847	2.936	1.0	24 / 3
10%, B=8, k=64	Torch CUDA	0.370	0.384	1.0	24 / 3

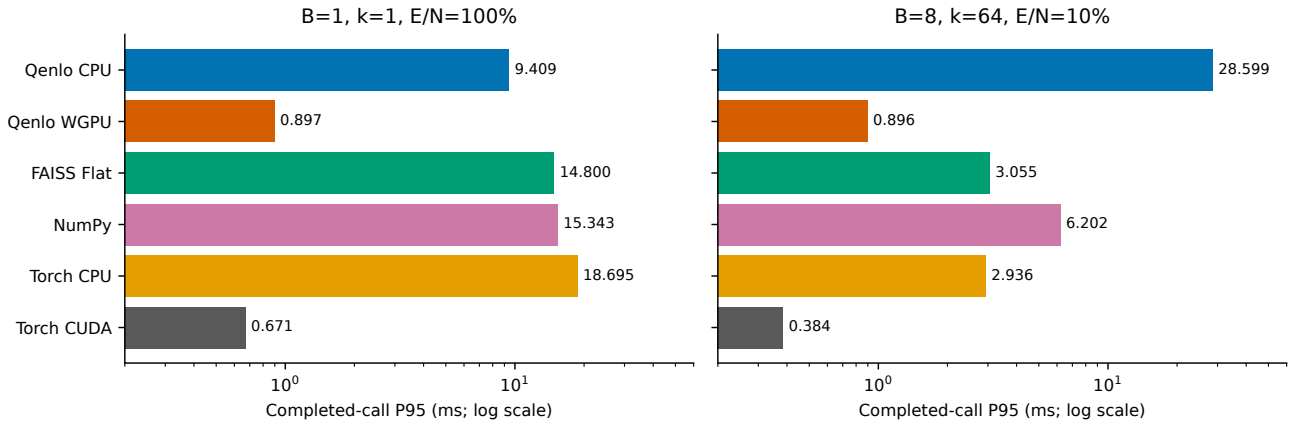

Figure 2: S2 `bb195fed`, RTX 4090/Vulkan, generated 100K×768. Host-call P95, logarithmic horizontal axes, no error bars. Eligibility/batching differ between panels. WGPU is the rejected candidate; all FP64 recalls 1.0. FAISS Flat, NumPy, Torch use prefiltered matrices.

Unfiltered batch-one WGPU is 10.49 times faster than native CPU at P95; Torch CUDA is 1.34 times faster than WGPU. At 10% eligibility and batch eight, these ratios are 31.91 and 2.33. They describe archived calls. Fresh matched runs are required to attach WGPU numbers to the final full-rescan selector. Tensor latency excludes deployment/persistence costs around a matrix.

6.3 Real embeddings

S1 real AG News has $N = 100\text{K}$, $D = 384$, $B = 16$, $k = 10$, $E/N = 1\%$. WGPU P95 is 0.171958 ms, CPU 0.941633 ms, FAISS Flat 0.242735 ms, NumPy 0.749214 ms, Torch CPU 0.268323 ms, Torch CUDA 0.351387 ms; all FP64 recalls are 1.0. WGPU leads Torch CUDA here, reversing their order in distinct S2 synthetic cells. This change across workload/source conditions does not isolate its cause. One fixture cannot establish robustness to models, geometry, or correlated metadata. ANN rows remain with recall in Appendix A.

7 Historical crossover and eligibility preparation

7.1 A localized exhaustive winner reversal

Matched H1 searches 100K×384 AG News on RTX 4050/DX12 at $B = 1, k = 10$. CPU takes 0.5198 ms P95 versus compact-row WGPU's 0.6340 ms at $E = 2\text{K}$. At 3K, WGPU takes 0.8144 ms versus CPU's 0.9563 ms and remains faster through 10K. The reversal is bracketed by $(2\text{K}, 3\text{K})$, not a fitted threshold. All localization recalls are 1.0. Compact-row timing includes duplicated count/materialize traversal.

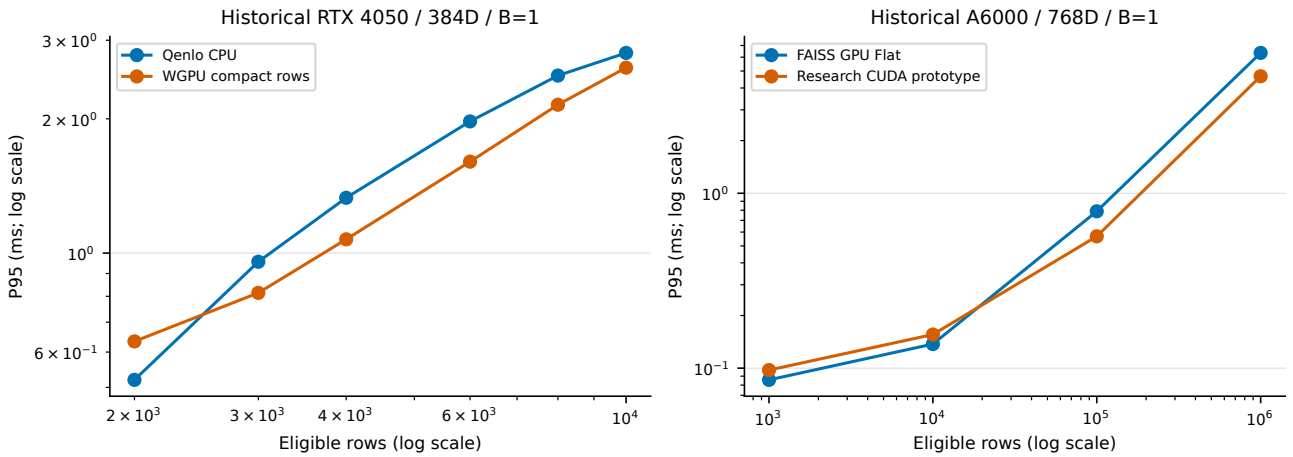

Figure 3: Separate historical panels. Left: H1 `3e2a4a9`, RTX 4050/DX12, AG News 100K×384, $B = 1, k = 10$, compact rows, median five-run P95, FP64 recall 1.0. Older endpoint revisions omitted. Right: H5 A6000 research CUDA/FAISS, generated 1M×768, prefiltered rows, $B = 1, k = 10$, P95 over 500 calls/system/cell; unordered top-10 agreement without independent FP64 oracle. Logarithmic axes; lines guide the eye, not fitted laws.

The shipped rule chooses CPU for $E < 4,096$ and batches below eight. Counterfactually applied to six H1 cells, it misroutes 3K and 4K, with relative P95 regret of 17.4% and 23.9%. For cell c and valid measured routes $\mathcal{R}_c$,

$$R(c) = \frac{T_{\pi(c),95}}{\min_{r \in \mathcal{R}_c} T_{r,95}} - 1. \tag{3}$$

These compare separate run-level percentiles, not paired queries. Equal weighting of six post-hoc cells is not an application distribution. A profile mechanism in code does not prove adaptive routing reduces regret.

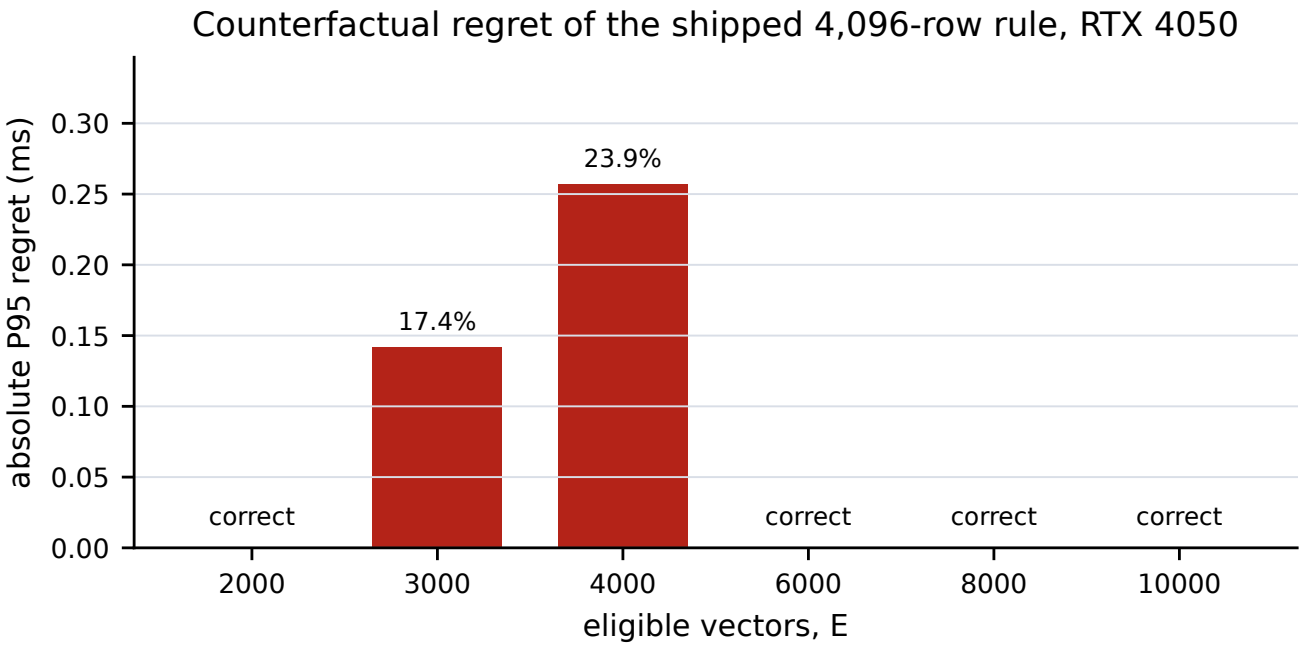

Figure 4: H1 RTX 4050, same 100K×384 compact-row cells and FP64 recall 1.0 as Figure 3. Completed-call P95 regret uses median five-run results and frozen 4,096-row rule. Annotations give relative regret. Descriptive counterfactual; no routing traces/confidence bars.

7.2 One-pass and cached eligibility

H2 uses a separate RTX 4090/Vulkan, Ryzen 9 7950X, driver 570.211.01. Its 100K×384 AG News ablation fixes scoring/queries across five runs per mode at $E \in \{1\text{K}, 4\text{K}, 6\text{K}, 100\text{K}\}$. One-pass lowers P95 from 0.131736 to 0.117870 ms at 1K, 0.305281 to 0.208009 at 4K, and 0.401130 to 0.271618 at 6K: 10.5–32.3% reductions. Warmed cache lowers sparse values by 24.3–57.3%. Dense improvement is smaller: 1.786170 to 1.720476 ms one-pass or 1.596024 cached.

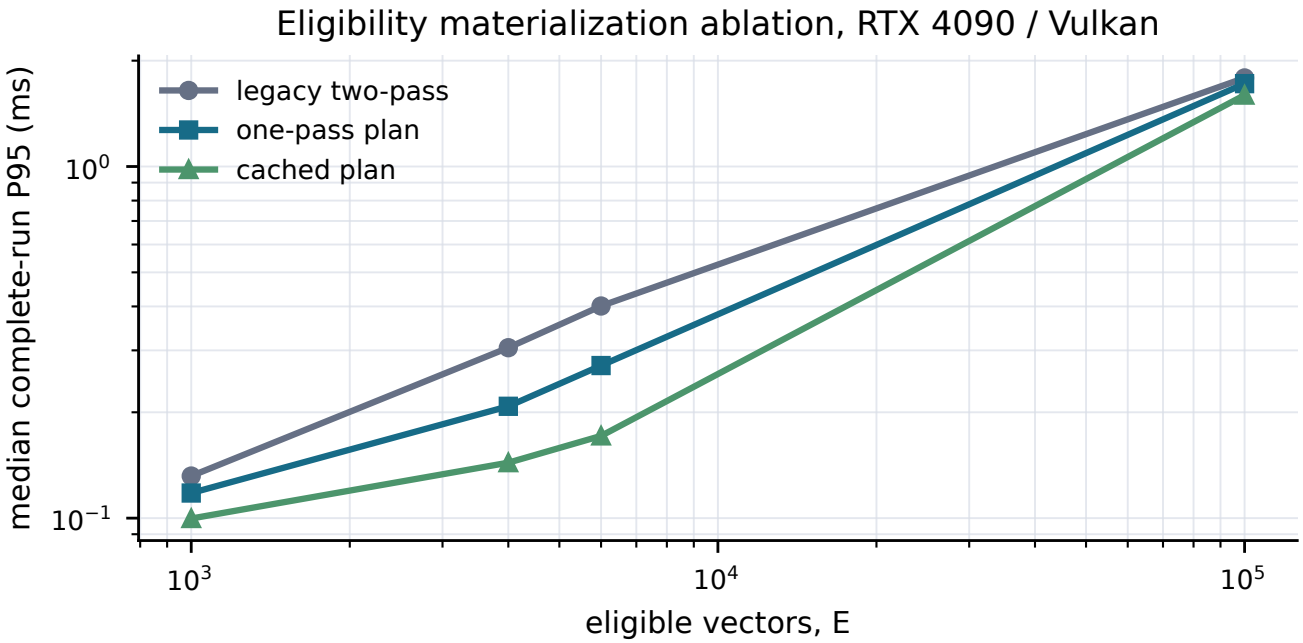

Figure 5: H2 `26e0ccc0`, RTX 4090/Vulkan, 100K×384 AG News, $B = 1, k = 10$. Median five-run completed-call P95, 25,000 calls/point. Sparse FP64 recall 1.0; dense 0.99998. Sequential modes, logarithmic axes, guide lines, no uncertainty intervals.

Cache benefit requires identical predicate/generation; mutation invalidates that premise. Without matched CPU cells, on another host, H2 cannot relocate H1's crossover. It directly establishes that duplicated eligibility work affects a completed call.

8 Predicate representation and batching

H0 dense GPU endpoints share a recorded GPU revision: shader predicates take 3.2404 ms P95, compact rows 4.2428, and masks 4.4591. At 1K eligible rows, compact rows take 0.6766 ms and shader predicates 2.8832 ms, but the sparse records differ in revision. That difference is context, not a causal representation ablation. The CPU and dense-GPU endpoints also differ. Figure 6 preserves these limitations rather than merging H0 with H1.

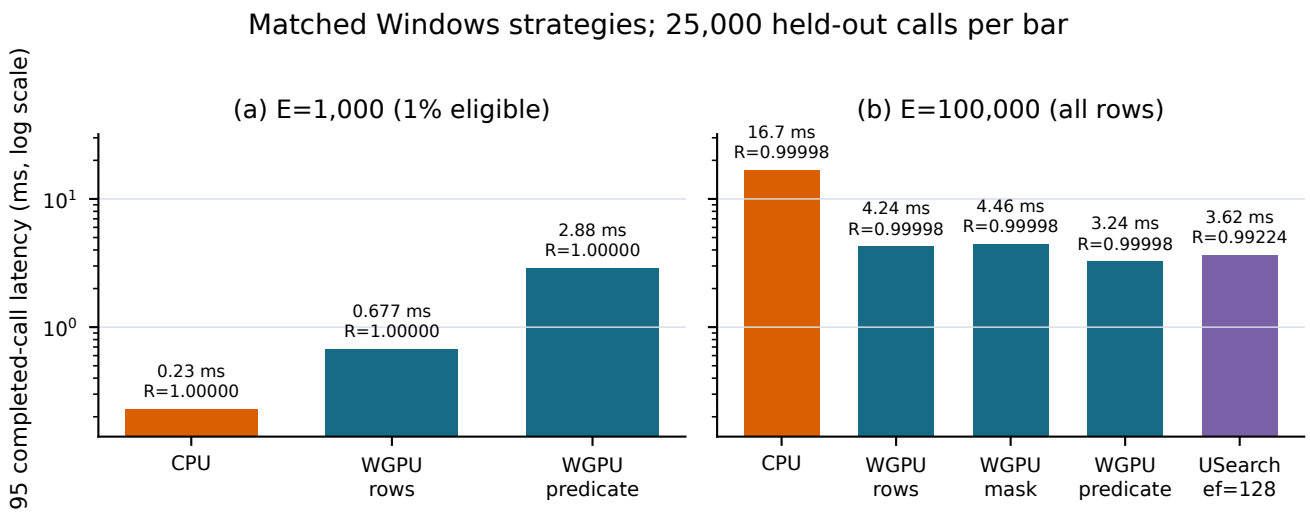

Figure 6: H0 RTX 4050/DX12, 100K×384 AG News, $B = 1, k = 10$, five runs/25,000 calls per row, completed-call P95. Dense WGPU forms share `f1ccc4d`; CPU/sparse records include other revisions. Exhaustive sparse/dense FP64 recall 1.0/0.99998; USearch ANN 0.99224. Descriptive context; no causal cross-revision speedup.

Batching affects dispatch amortization and selection; representation affects host preparation and device work. C1–C6 supports both as routing features but changes them together with dimension and selectivity. Device-lab batches use another reporting schema. No same-host factorial study isolates all interactions, and no fixed-E, varying-N study establishes eligibility as a statistically superior predictor to size.

9 ANN and external-engine comparisons

9.1 Chroma qualification and historical competitors

Seven qualified S1 pairs favor candidate WGPU over Chroma HNSW, with P95 ratios 3.700–3233.136. All seven paired Chroma recalls are 1.0. The maximum is real AG News: Chroma 555.964 ms versus WGPU 0.171958. Three attempts fail the gate before held-out timing: C3/C6 at expansion 128 and deep D2 at 512. Completed USearch D2 has recall 0.3453125 at 83.3133 ms and is unqualified. These rows remain visible.

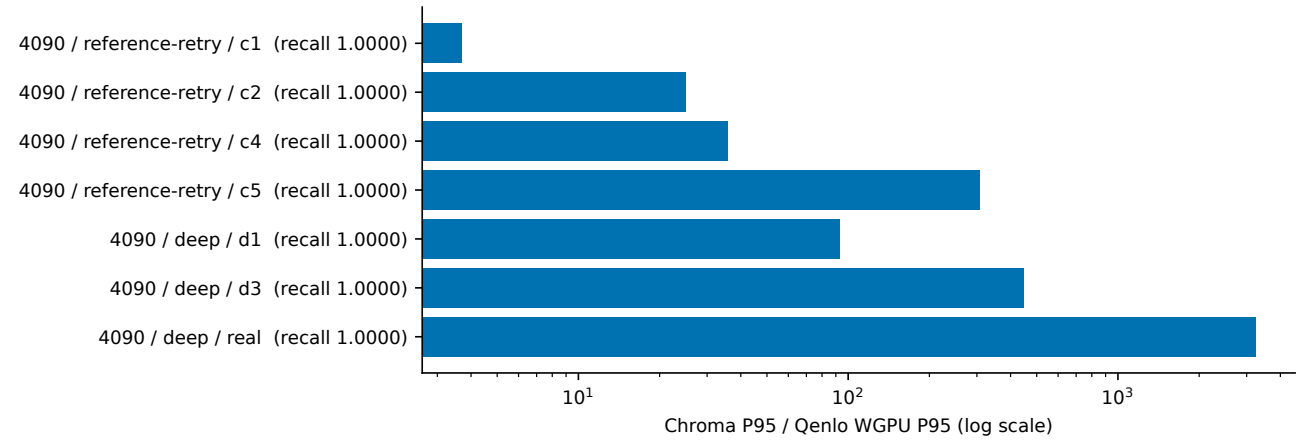

Figure 7: S1 `730c0500`, seven qualified configuration/workload pairs on A40/RTX 4090, $k = 10$; IDs map to matrix/appendix. Logarithmic axis: Chroma/native candidate-WGPU completed-call P95 ratio. Chroma recall printed. Python API, ANN, persistence/filter boundaries differ. Three gate failures excluded from ratios, retained in ledger; no error bars.

H0 Chroma dense reaches 1.1303 ms P95 at recall 0.99414 versus WGPU predicate 3.2404 at 0.99998. Tuned USearch expansion 128 takes 3.6245 at 0.99224. These are operating points with revision/API qualifications, not ANN frontiers. H3 Linux favors FAISS HNSW at 1.4939 ms/0.9994 over Qenlo CPU 12.9014/1.0. Chroma, USearch, FAISS Flat, Milvus Lite, and LanceDB remain in the appendix. These boundaries preclude universal rankings.

9.2 A6000 real and synthetic evidence

H4 strict TopK yields $E = 10,026$ from 1M 768D rows. FAISS GPU Flat leads qualified methods at 0.132222 ms P95, then research CUDA prototype 0.170722 and NumPy 0.197311. Each has 5,000 calls and FP64 recall 1.0. cuVS's 0.384636 ms is disqualified by recall 0.9. The prototype misses the retained gate requiring twofold CPU advantage. Eligibility is pre-materialized; the prototype is Python/PyTorch CUDA, not shipped Rust/WGPU. H5 fresh-seed synthetic differs: FAISS leads at $E = 1K/10K$; the buffered prototype leads at 100K (0.5673 versus 0.7887 ms) and 1M (4.6683 versus 6.3475). The reversal is between measured 10K/100K points. Correctness is unordered top-10 agreement, not an FP64 oracle. This is formulation-specific. Native WGPU had no usable A6000 Vulkan ICD and no native timing in that cohort.

10 Mutation, reopen, and resource accounting

S1 lifecycle uses a durable 10K×384 collection on RTX 4090, $B = 1, k = 10$, unfiltered. First-search P95 after add-one, delete-one, add-batch, delete-batch is 0.598, 0.631, 0.740, 0.819 ms. Mutation is excluded and has separate P95 near 22 ms. Three observations/type report zero rebuilds; immediate-deletion checks pass. Reopen takes 52.169 ms to open and, separately, 101.296 ms for first search with one rebuild. Neither is a combined timer. One observation cannot characterize reopen tail latency.

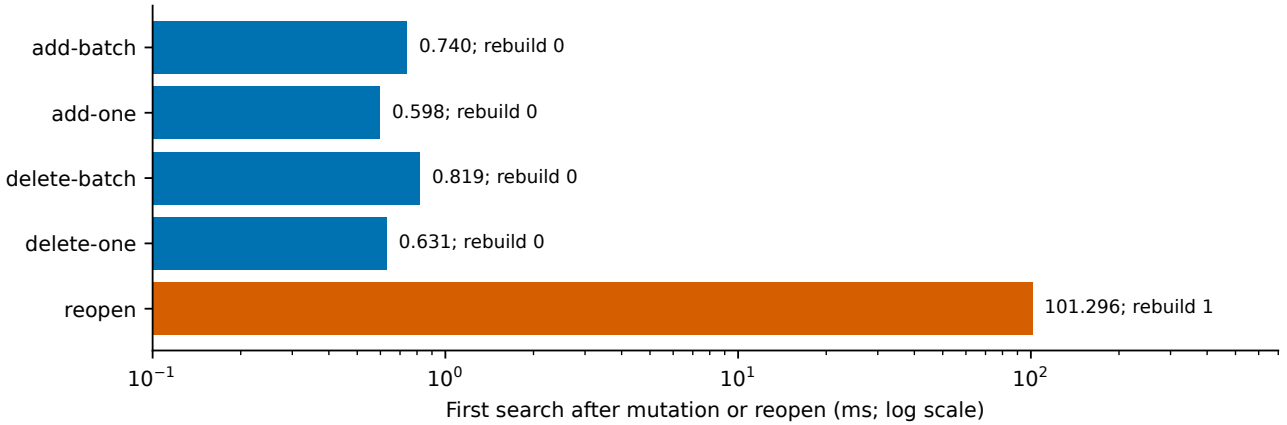

Figure 8: S1 reference RTX 4090, durable 10K×384, unfiltered $B = 1, k = 10$. Bars are first-search latency after mutation/reopen, excluding mutation/opening. Three observations per mutation type give P95; reopen is one observation. Rebuild annotations are fractions, not probabilities. Deletion checks pass. Logarithmic horizontal axis, no error bars.

S2 unfiltered WGPU reports 311.181 MB of owned accelerator allocation and a 1.203675 GB process RSS high-water mark. Batch eight reports 314.909 MB and 1.174577 GB, respectively. Units are decimal. RSS includes process and runtime peaks; owned buffers omit driver and runtime allocations. Neither is physical VRAM telemetry. Upload and readback counts are 403,136 and 48 bytes unfiltered, and 517,120 and 65,792 bytes for batch eight. Residency and per-call transfer are separate scopes.

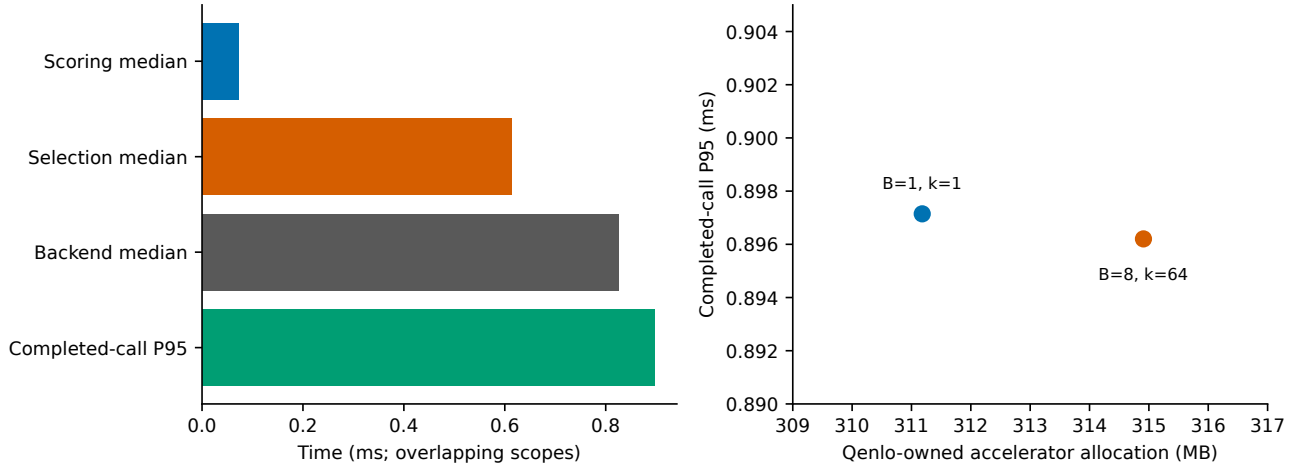

Figure 9: S2 `bb195fed`, RTX 4090/Vulkan, generated 100K×768, FP64 recall 1.0. Left: $B = 8, k = 64, E/N = 10\%$, raw phase medians versus call P95. Nested scopes are not additive. Right: two corrected WGPU workloads, zoomed linear latency axis, decimal owned-allocation MB. Different workloads do not establish a memory/latency law or efficiency score. No error bars.

Batch-eight median scoring is 0.072 ms, selection 0.613, backend execution 0.826; call P95 is 0.896. Selection exceeds scoring in these diagnostics. That motivates investigation but does not prove another selector improves latency. Capacity, initialization, and warm-state validity remain deployment costs even when arithmetic is inexpensive.

11 Mobile and heterogeneous devices

H6 records physical Android: Android 16 (API 36), MediaTek MT6897, Mali-G615 MC6 with Vulkan, app 0.1.0. All 21 aggregate cells report recall@10=1.0 without fallback or failure. CPU wins the 10K quick batch-one cell (5.767 versus WGPU's 12.487 ms P95); WGPU wins the 100K full cell (16.975 versus 29.436) and soak cell (17.678 versus 27.019). Size and suite change together, so this is a qualitative reversal rather than a mobile crossover bracket.

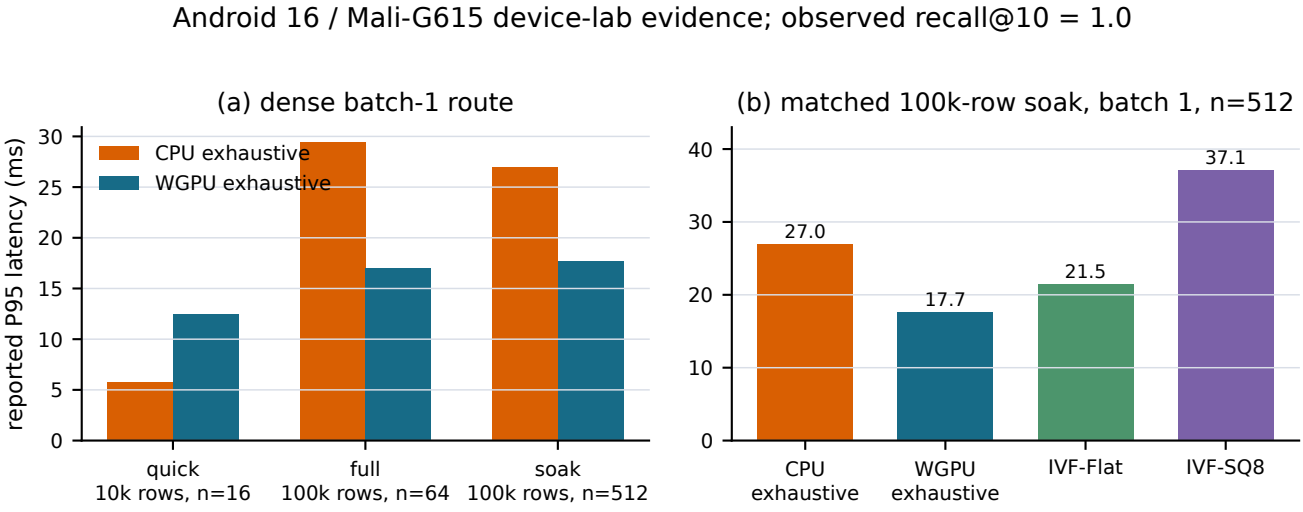

Figure 10: H6 physical Mali-G615/Vulkan, generated 384D, $k = 10$, app 0.1.0; commit unrecorded. Supplied aggregate percentiles: quick/full/soak use 16/64/512 samples/cell. All recalls 1.0; IVF remains approximate. Raw calls and controlled thermal/power records absent. No error bars or current-revision mobile claim.

At 100K Android soak, IVF-Flat takes 21.453 ms P95 and IVF-SQ8 37.090, slower than exhaustive WGPU at those settings. Dense WGPU batch eight reports 11.131 ms; the batch statistic is not per-query normalized. It suggests amortization, not an eightfold per-query gain. Power source is absent and thermal state is uninterpreted "0".

H7 Intel Arc soak reports CPU 16.486 ms P95, exhaustive WGPU 4.444, IVF-Flat 2.704, IVF-SQ8 22.797 at 100K×384. All recalls are 1.0 over 512 samples/cell. Arc adds non-NVIDIA execution evidence and another negative for universal exhaustive dominance: IVF-Flat wins. One externally labeled "full" report embeds `suite=quick`; JSON is authoritative. Historical Android runs and Kotlin/JVM bindings do not establish current Android packaging. Simulator/macOS builds do not establish physical iOS performance; no iOS-device/MPS timing is retained.

12 Negative results and rejected optimization

Lane-minimum selection retains each lane's minimum and rescans only the winning lane. It reduces repeated scans but changes local state and synchronization. Twelve qualified S0/S1 pairs yield five wins and seven losses, with frozen-baseline to candidate P95 ratios from 0.617 to 1.295. The retained implementation uses the simpler full-rescan selector. Less nominal work did not consistently reduce completed latency. Archives contain other changes too: paired revisions do not prove every difference is solely caused by selection.

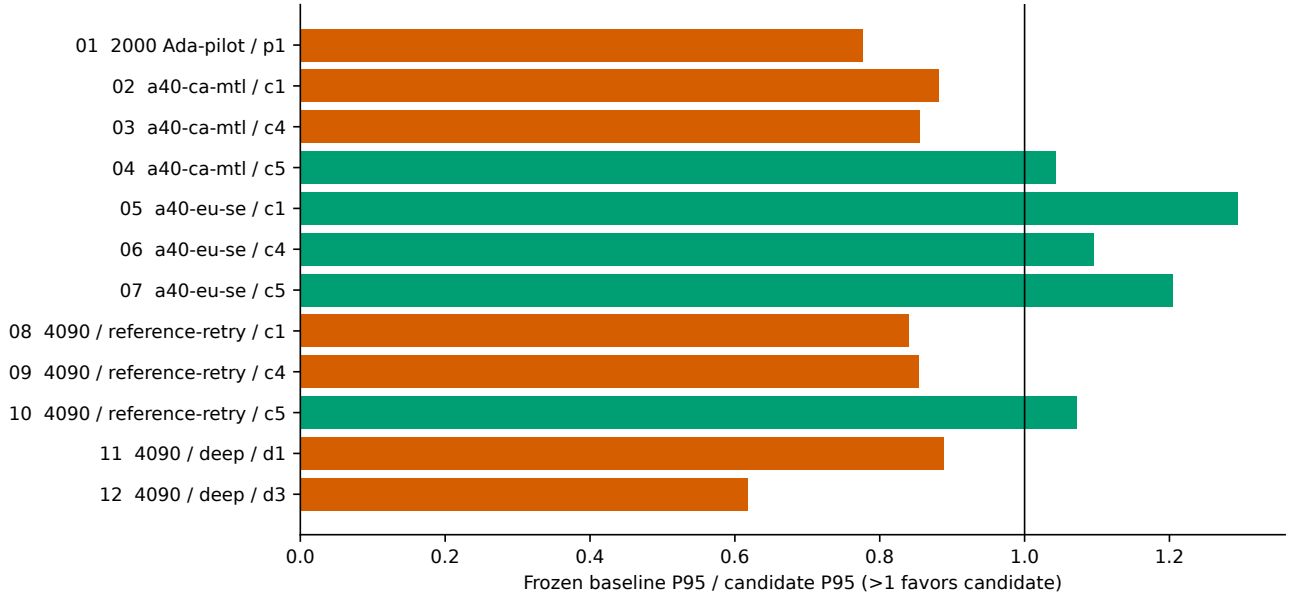

Figure 11: S0 `f37e744c` versus S1 `730c0500`, twelve qualified WGPU configuration/workload pairs, FP64-oracle-qualified completed-call P95. Exact workloads/ratios in appendix. Above one favors candidate: five wins, seven losses. RTX 2000 Ada, A40, RTX 4090; no significance test, error bars, or pooled device effect.

Four pre-fix 100K×768 failures remain: a 2 GiB benchmark budget did not override 512 MiB default in-memory admission. Corrected source/results have distinct hashes, not retroactive qualification. Two malformed workload-label invocations are invalid harness rows, not slow engines. Chroma gates, unqualified USearch, cuVS recall/runtime failures, and unavailable Vulkan paths remain evidence. Controller completion alone is insufficient qualification.

Final known daily Runpod spend is $0.9400778694252952, with zero campaign pods remaining, ten creations and ten verified deletions. No campaign volume was retained. This is account/day billing, not per-query or clean marginal campaign cost; billing lag is possible. No cloud work was rerun for authoring.

13 Deployment implications

CPU fits tiny eligible work where accelerator fixed costs dominate. WGPU can help dense/batched work with valid residency. Tensor execution may fit applications already owning its runtime, but package footprint and persistence-integration cost were not measured. Snapshot generation/mutation state must be checked before amortizing preparation. An explanatory decomposition is

$$T_{\text{call}} = T_{\text{prepare}} + T_{\text{route}} + T_{\text{execute}} + T_{\text{return}}. \tag{4}$$

These conceptual nonoverlapping boundaries are not sums of overlapping counters. Execution depends on E, D, B, k, representation, and device; preparation also depends on reuse, generation, residency. No coefficients are fitted. This organizes observations without a transferable latency law. The reversals and preparation ablation support the central engineering inference: size/GPU presence is insufficient for routing. They do not validate a multidimensional predictor. Decisions require the named adapter/API, recall and mutation conditions. Cloud results cannot select a mobile default.

14 Threats to validity

The archive grew sequentially, not as a randomized factorial study. Dirty trees limit attribution despite hashes/configurations. H0 is cross-revision context; H1 is post-hoc, one laptop/dimension/batch/corpus, independent synthetic metadata. H2 is sequential, lacks CPU cells, and cannot recalibrate H1. C1–C6 confound size, dimension, batching, eligibility. S2 uses a rejected selector whose final replacement was not rerun in corrected cells. Three-run small, five-run historical, and single-reopen data do not support narrow intervals or broad significance. Repeated queries are not independent workloads. Aggregate device data cannot yield raw distributions; thermal/power controls are incomplete. Prototype agreement is weaker than an FP64 oracle. Reference checks do not prove all ties or workloads agree. Adapters differ in filtering, selection, normalization, threading, persistence, API. NumPy uses full lexicographic sorting, not an optimal top-k baseline. Corrected FAISS is CPU Flat; historical A6000 is GPU Flat. Memory scopes differ and omit peak total accelerator use. Concurrency, energy, device loss, cache churn, adversarial predicates, and crash schedules were not benchmarked. Native sweeps lack ANN frontiers/filter-aware baselines. No fastest-database claim follows.

15 Related work and novelty boundary

FAISS describes optimized exhaustive/approximate CPU/GPU search and selection [3, 6]. HNSW provides graph ANN [8]. DiskANN/SPANN address storage-aware regimes [2, 12]; Filtered-DiskANN/ACORN modify graph access under predicates [4, 10]. Filtered-ANN studies establish that attributes, selectivity, query/data conditions, tuning, and implementation affect method choice [5, 7, 11]. System analyses examine filtering plans and approximate/exhaustive alternatives [1]; the latter two studies are preprints. "Selective scans sometimes win" is not novel. Qenlo's narrower contribution is the CPU/portable-GPU completed-call boundary, eligibility compilation and generation-bound state, and small-collection resource/lifecycle evidence. It lacks broad FANNS coverage and claims no new ANN algorithm.

16 Conclusions

A matched RTX 4050 sweep localizes CPU/WGPU reversal and static-policy regret. A separate ablation shows eligibility preparation changes completed latency. Small-collection results add fast experimental WGPU/tensor paths, CPU wins on selective cells, resource costs, and a rejected selector. Android, Arc, A6000 extend observations within separate correctness/platform boundaries. Routing should account for eligibility, representation, batching, runtime, state. Evidence does not establish a universal threshold, numerical-equivalence theorem, or adaptive router's held-out advantage. Final local selection needs new matched measurements before inheriting corrected experimental latencies.

Reproducibility statement

Retained evidence only: no product/benchmark changes or experimental reruns. Ledgers preserve claims, revisions/hashes, environments, provenance, timing, counts, correctness, memory, qualification, exclusions. Paper-only scripts regenerate reductions/tables and PDF/PNG figures without replacing raw evidence or historical figures. Appendix B and `paper/REPRODUCE.md` describe audit/clean build.

References

[1] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis. *arXiv preprint arXiv:2602.11443*, 2026. URL https://arxiv.org/abs/2602.11443.

[2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In *Advances in Neural Information Processing Systems*, volume 34, pages 5199–5212, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html.

[3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. URL https://arxiv.org/abs/2401.08281.

[4] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pages 3406–3416. Association for Computing Machinery, 2023. doi: 10.1145/3543507.3583552. URL https://doi.org/10.1145/3543507.3583552.

[5] Patrick Iff, Paul Brügger, Marcin Chrapek, David Kochergin, Maciej Besta, and Torsten Hoefler. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 610–622, 2026. doi: 10.1145/3805712.3809731. URL https://doi.org/10.1145/3805712.3809731.

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572. URL https://doi.org/10.1109/TBDATA.2019.2921572.

[7] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proceedings of the ACM on Management of Data*, 3(6):298:1–298:26, 2025. doi: 10.1145/3769763. URL https://doi.org/10.1145/3769763.

[8] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42 (4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

[9] Beatrix Miranda Ginn Nielsen. DTU pretrained sentence BERT AG News embeddings, 2023. URL https://doi.org/10.11583/DTU.21286923.v1.

[10] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3): 120:1–120:27, 2024. doi: 10.1145/3654923. URL https://doi.org/10.1145/3654923.

[11] Jiayang Shi, Yuzheng Cai, and Weiguo Zheng. Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study. *arXiv preprint arXiv:2509.07789*, 2025. URL https://arxiv.org/abs/2509.07789.

[12] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html.

[13] TopK. TopK Bench: Vector database benchmarks. https://github.com/topk-io/bench, 2026. Commit 398ce7d72dea7ad765ada54c5daa014c498efb52; accessed 2026-09-05.

[14] W3C GPU for the Web Working Group. WebGPU. https://www.w3.org/TR/webgpu/, 2026. W3C Candidate Recommendation Draft, 20 August 2026; accessed 2026-09-05.

[15] W3C GPU for the Web Working Group. WebGPU Shading Language. https://www.w3.org/TR/WGSL/, 2026. W3C Candidate Recommendation Draft, 17 August 2026; accessed 2026-09-05.

A Detailed retained tables

These tables are generated from retained machine-readable evidence. They preserve cohort-specific aggregation; an empty field is not a zero. The full 182-row campaign CSV/JSON includes every failed, unavailable, invalid, and unqualified row. Native versus replay sample units are reconciled in the main table. All latency units below are milliseconds.

A.1 Common small-collection native rows

Table 4: S1 common cells from the matrix. CPU values on A40 are absent for this source role; no substitute baseline-CPU row is silently inserted. Workload definitions appear in the main text. All displayed rows qualify at FP64 recall 1.0.

Configuration	Cell	CPU P95	WGPU P95
a40-ca-mtl	C1	–	0.1595
a40-ca-mtl	C2	–	0.2398
a40-ca-mtl	C3	–	0.4277
a40-ca-mtl	C4	–	0.1514
a40-ca-mtl	C5	–	0.1590
a40-ca-mtl	C6	–	13.7685
a40-eu-se	C1	–	0.1779
a40-eu-se	C2	–	0.3574
a40-eu-se	C3	–	0.4330
a40-eu-se	C4	–	0.1582
a40-eu-se	C5	–	0.1666
a40-eu-se	C6	–	13.7465
4090 EU-RO	C1	0.0143	0.0943
4090 EU-RO	C2	0.0484	0.1347
4090 EU-RO	C3	1.9988	0.2695
4090 EU-RO	C4	0.0082	0.0911
4090 EU-RO	C5	0.0341	0.0953
4090 EU-RO	C6	68.0575	4.0479

A.2 Frozen-baseline versus candidate selection

Table 5: All twelve qualified S0/S1 pairs; ratio is frozen baseline P95 divided by candidate P95. IDs encode rows (r), dimension (d), batch (b), k when nondefault, and eligible fraction (f). The compound cell combines metadata clauses.

Configuration	Workload	Ratio
rtx2000-eu-ro-pilot	p1-r1000-d128-b1-f1	0.777
a40-ca-mtl	c1-r1000-d128-b1-f1	0.881
a40-ca-mtl	c4-r10000-d384-b1-f001	0.855
a40-ca-mtl	c5-r100000-d128-b1-f001	1.043
a40-eu-se	c1-r1000-d128-b1-f1	1.295
a40-eu-se	c4-r10000-d384-b1-f001	1.096
a40-eu-se	c5-r100000-d128-b1-f001	1.205
4090 / reference-retry	c1-r1000-d128-b1-f1	0.840
4090 / reference-retry	c4-r10000-d384-b1-f001	0.853
4090 / reference-retry	c5-r100000-d128-b1-f001	1.073
4090 / deep	d1-r5000-d384-b8-k10-f01	0.889
4090 / deep	d3-r50000-d384-b1-k10-compound	0.617

A.3 Small real-embedding replay

Table 6: S1 AG News, reference RTX 4090, 100K×384, $B = 16, k = 10, E/N = 1\%$. Three runs; 939 calls per native/replayed engine. Chroma and USearch are ANN; all others exhaustive. External flat/tensor matrices are prefiltered.

Engine	Completed-call P95	FP64 recall
chroma-ef512	555.9637	1.00000
current-cpu	0.9416	1.00000
current-gpu	0.1720	1.00000
faiss-flat	0.2427	1.00000
numpy	0.7492	1.00000
torch-cpu	0.2683	1.00000
torch-cuda	0.3514	1.00000
usearch	294.6324	0.99994

A.4 Historical Windows endpoints and matched localization

Table 7: H0: 100K×384, $B = 1, k = 10$, five runs/25,000 calls per row. Cross-revision endpoint context: dense CPU `4ae9f1d`, dense WGPU `f1ccc4d`; sparse CPU/rows `78e3947`, sparse predicate `f1ccc4d`. ANN rows retain their own revisions/API.

Eligible	System	P95	FP64 recall
1000	CPU exhaustive	0.2304	1.00000
1000	WGPU compact rows	0.6766	1.00000
1000	WGPU predicate	2.8832	1.00000
100000	CPU exhaustive	16.6567	0.99998
100000	WGPU compact rows	4.2428	0.99998
100000	WGPU mask	4.4591	0.99998
100000	WGPU predicate	3.2404	0.99998
100000	USearch HNSW ef=128	3.6245	0.99224
100000	Chroma HNSW ef=128	1.1303	0.99414

Table 8: H1: matched recorded revision `3e2a4a9`, RTX 4050/DX12, AG News, 100K×384, $B = 1, k = 10$, five runs/25,000 calls per backend/cell. Post-hoc localization; median run P95.

Eligible	CPU P95	Compact WGPU P95	FP64 recall
2000	0.5198	0.6340	1.00000
3000	0.9563	0.8144	1.00000
4000	1.3305	1.0737	1.00000
6000	1.9735	1.6033	1.00000
8000	2.5003	2.1508	1.00000
10000	2.8104	2.6041	1.00000

A.5 Historical eligibility ablation

Table 9: H2, RTX 4090/Vulkan, 100K×384, $B = 1, k = 10$. Five runs/25,000 calls per mode/cell, sequential mode execution. Percentiles are medians of run percentiles.

Eligible	Preparation	P50	P95	P99	FP64 recall
1000	legacy-two-pass	0.1204	0.1317	0.1584	1.00000
1000	one-pass	0.1097	0.1179	0.1453	1.00000
1000	cached	0.0908	0.0997	0.1132	1.00000
4000	legacy-two-pass	0.2910	0.3053	0.3251	1.00000
4000	one-pass	0.1982	0.2080	0.2250	1.00000
4000	cached	0.1348	0.1438	0.1582	1.00000
6000	legacy-two-pass	0.3851	0.4011	0.4400	1.00000
6000	one-pass	0.2561	0.2716	0.2903	1.00000
6000	cached	0.1624	0.1713	0.1941	1.00000
100000	legacy-two-pass	1.6633	1.7862	1.9031	0.99998
100000	one-pass	1.6074	1.7205	1.8165	0.99998
100000	cached	1.5379	1.5960	1.6907	0.99998

A.6 Historical A6000 distributions

Table 10: H4 strict real TopK, 1M×768, $E = 10,026$, $B = 1, k = 10$, 5,000 calls/system. Shared prefiltered matrix, host completed calls. cuVS fails correctness and is excluded from qualified comparisons. CUDA prototype is unshipped research code.

System	P50	P95	P99	FP64 recall
cuVS (disqualified)	0.3615	0.3846	0.4131	0.90000
FAISS GPU Flat	0.1261	0.1322	0.1458	1.00000
NumPy CPU exhaustive	0.1217	0.1973	0.2283	1.00000
CUDA prototype	0.1600	0.1707	0.1809	1.00000

Table 11: H5 generated normalized FP32, 1M×768, $B = 1, k = 10$, 500 calls/system/cell across five runs. Completed-call distribution percentiles; unordered top-10 agreement, not independent FP64 truth.

Eligible	System	P50	P95	P99
1000	FAISS GPU Flat	0.0672	0.0857	0.0952
1000	CUDA prototype	0.0775	0.0975	0.1155
10000	FAISS GPU Flat	0.1272	0.1373	0.1505
10000	CUDA prototype	0.1438	0.1557	0.1741
100000	FAISS GPU Flat	0.7755	0.7887	0.8001
100000	CUDA prototype	0.5559	0.5673	0.5763
1000000	FAISS GPU Flat	6.2736	6.3475	6.4357
1000000	CUDA prototype	4.6508	4.6683	4.7364

A.7 Historical Linux libraries

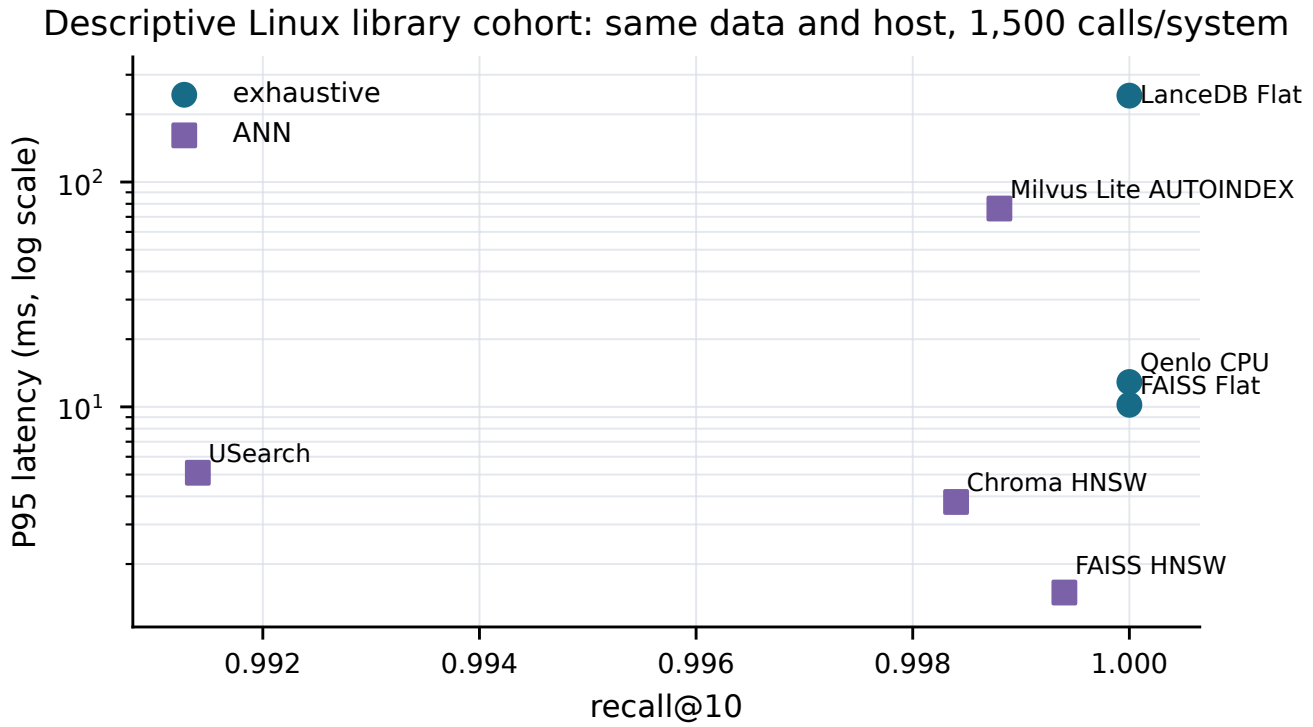

Figure 12: H3 `bb51987`, Linux CPU library context, separately prepared AG News 100K×384, unfiltered $B = 1, k = 10$. Three runs/1,500 calls per system; completed library-call P95. Recall accompanies latency; algorithms, persistence, and APIs differ. This figure is descriptive within the host, not a product leaderboard.

Table 12: H3 complete retained library context. ANN recall is part of each operating point; exhaustive engines use different APIs. Native GPU failed Vulkan initialization and has no timed row.

System	Search	P95	FP64 recall
Qenlo CPU	exhaustive	12.9014	1.00000
USearch	ANN	5.0933	0.99140
FAISS Flat	exhaustive	10.2171	1.00000
FAISS HNSW	ANN	1.4939	0.99940
Chroma HNSW	ANN	3.7767	0.99840
Milvus Lite AUTOINDEX	ANN	76.2294	0.99880
LanceDB Flat	exhaustive	242.2557	1.00000

A.8 Android aggregate records

Table 13: H6 all 21 supplied Android cells. $D = 384, k = 10$; B is batch size, n is reported samples. All recall@10 values are 1.0, all fallback counts zero. Automatic rows use selective predicates and are not comparable with dense rows. No raw-call reconstruction is possible.

Suite	Cell	Rows	B	n	P50	P95	P99
quick	cpu-exact-all	10000	1	16	1.8410	5.7670	5.7670
quick	gpu-exact-all	10000	1	16	8.7070	12.4870	12.4870
quick	gpu-native-batch-8	10000	8	16	5.8860	5.8860	5.8860
quick	gpu-ivf-flat-recall-95	10000	1	16	3.6160	8.0560	8.0560
quick	gpu-ivf-sq8-recall-95	10000	1	16	4.7460	7.3160	7.3160
quick	automatic-selective-cpu-route	10000	1	16	0.0230	0.0250	0.0250
quick	automatic-selective-batch-8	10000	8	16	2.3100	2.3100	2.3100
full	cpu-exact-all	100000	1	64	18.3080	29.4360	41.5590
full	gpu-exact-all	100000	1	64	13.8060	16.9750	48.3750
full	gpu-native-batch-8	100000	8	64	10.6080	11.1010	11.1010
full	gpu-ivf-flat-recall-95	100000	1	64	8.8510	21.6170	32.4310
full	gpu-ivf-sq8-recall-95	100000	1	64	20.3940	26.5780	29.1260
full	automatic-selective-cpu-route	100000	1	64	0.2560	0.3110	0.3230
full	automatic-selective-batch-8	100000	8	64	10.0390	15.5710	15.5710
soak	cpu-exact-all	100000	1	512	25.7990	27.0190	27.5720
soak	gpu-exact-all	100000	1	512	14.2740	17.6780	25.7690
soak	gpu-native-batch-8	100000	8	512	10.2900	11.1310	12.3880
soak	gpu-ivf-flat-recall-95	100000	1	512	13.9770	21.4530	24.5490
soak	gpu-ivf-sq8-recall-95	100000	1	512	27.3590	37.0900	40.5810
soak	automatic-selective-cpu-route	100000	1	512	0.3570	0.5440	0.7430
soak	automatic-selective-batch-8	100000	8	512	8.1740	9.8900	12.4410

B Claim ledger and provenance

The definitive machine-readable ledger is `paper/tables/claim-to-artifact.json`, with a CSV index at the same stem. It incorporates audited historical, campaign, and system claims and every matrix row. Each measurement records source, revision/hash, environment, data provenance, workload, filter, timing, sample/repetition unit, oracle, memory scope, qualification, and limitations. Unknown provenance is explicit rather than inferred from a nearby cohort. Full machine paths remain outside narrow printed tables for legibility.

Table 14: Headline claim families and authoritative artifacts. Paths are relative to the repository; complete hashes and per-row fields are in the ledger.

Claim	Result / qualification	Artifact family
H1/H1-regret	2K–3K reversal; up to 23.9% counterfactual regret, one historical laptop	`research/data/raw/2026-09-02-native-crossover/`; per-cell samples/runs/configurations
H0	Historical representation/endpoints; different revisions explicitly separated	`benchmarks/2026-08-28/real/`
H2	One-pass/cache preparation; five sequential runs, no CPU recalibration	`research/artifacts/qenlo-phase1-eligibility-2026-09-04.tar.gz`
S2	Twelve corrected 768D rows, recall 1.0; candidate selector	Campaign `deep768/rtx4090-eu-ro-secure-deep768-admission-fix/`
S1-selector	Five wins/seven losses; rejected unconditional optimization	Matrix paired `baseline-gpu/current-gpu`; source archives
S1-real/ANN	Real fixture; seven qualified Chroma pairs; failed gates and USearch visible	Campaign `deep/` and `reference/`; matrix and raw samples
S1-lifecycle	Search-only P95; separate mutation/opening; zero/final-one rebuilds	Reference `lifecycle-current-gpu` samples/summary
S2-resources	Device allocations, process RSS, transfers, nested phases	Corrected native samples, process time logs, matrix

Table 15: Historical, platform, accounting, and semantic claim families (continued).

Claim	Result / qualification	Artifact family
H4/H5	Real strict negative; synthetic formulation reversal	`benchmark-results/2026-09-02-runpod-a6000-strict-research-gate/`; `research/data/raw/2026-09-02-a6000-cuda/heldout/`
H3	Seven CPU library rows, no WGPU timing	`benchmark-results/2026-09-02-runpod-archive/retained-archive/artifacts/`
H6/H7	Physical Android and Arc, aggregate-only	`research/data/raw/2026-09-02-android/`; `benchmarks/2026-08-31/device-lab/intel-arc/reports.json`
Accounting	Account daily spend, zero retained campaign resources	Campaign `ledger.jsonl`, billing captures, `campaign-summary.json`
Semantics	Current source contracts; not retrospective performance	`paper/audit/system-ledger.json`

B.1 Source and result archives

The campaign root is `research/artifacts/runpod-small-2026-09-05/`. Full identities are:

S0 frozen source. `f37e744c7ee4054a74c4b4181f2dde61dade4d677bbc5493b9a36b61d165a45d`

S1 experimental source. `730c050065e6786972a7be4f9bcb619168becdc943ce1122d1583c61f66409e3`

S2 corrected source. `bb195fedb6519c26598c6c103e7e182982c2eddd0a06d9c2cf54eb5cb5ceeb19`

S2 result archive. `9e322d615c0bbed44616faa831493ca030dd49bbce7fe0c4386f49e3b8532fc0`

H2 eligibility archive. `26e0ccc057c59c0fb4687f8d85a923e88b7bb25257e4e551e22cffe13c1b821f`

S2 result integrity is distinct from S2 source identity. The corrected result manifest has 74 non-self entries. The full campaign verifier checks eight retained result archives and nine checksum manifests, 1,053 entries, skipping four self-manifest entries. It also checks source roles and the local/baseline WGSL identity. Checksums establish byte integrity, not scientific qualification.

B.2 Discrepancies resolved during authoring

The audit reconciled disagreements instead of choosing flattering prose: (i) "current" source archives still contain the rejected selector; (ii) the matrix lifecycle scope string is broader than the separate raw timers; (iii) external replay preparation omits eligibility traversal despite its field name; (iv) completed is not identical to qualified; (v) historical endpoint revisions differ; (vi) Intel Arc's embedded quick label overrides an external full label; and (vii) final spend is account/day accounting. Bibliographic corrections include the DTU dataset's publication year, the SPANN title, DiskANN author spelling, and the full Filtered-DiskANN author list. Audit notes retain these resolutions.

B.3 Excluded evidence and unresolved claims

No raw evidence or old figure was deleted. The historical inventory hashes every top-level figure file; regenerated figures live in a separate `paper/figures/final/` directory. Older dimension, matched-size, metadata-correlation, and heatmap graphics do not establish unmeasured experiments. Redundant crossover/selectivity plots are superseded by explicitly scoped panels. The old architecture figure is retained, while the paper explains canonical state in equations and prose.

An incomplete historical CPU development run and duplicate cloud mirrors are excluded from counts. H3 native Vulkan initialization failed; there is no GPU latency to rank. Historical Qdrant was deliberately outside the retained comparison because its local-mode search boundary differed; no conclusion about its performance follows. Provider-compatible TopK experiments used an inclusive predicate yielding 10,148 eligible rows and supplied truth, versus strict H4's 10,026 rows and independent oracle. They remain exploratory negative context

and are not pooled with H4. Their approximately-one-percent prototype did not beat FAISS; other provider cells had correctness disagreements. cuVS numerical/runtime failures remain failures, not qualified exact results.

The archive does not test a complete multidimensional routing surface, fixed-E varying-N, controlled metadata/vector correlation, a tuned filtered-ANN frontier, calibrated held-out routing, concurrency, energy, total VRAM peaks, long-running mutation churn, arbitrary crash recovery, MPS, or physical iOS. Historical mobile results cannot certify the final revision. S2's final-local-selector rerun is absent. These gaps constrain claims rather than licensing extrapolation.

B.4 Reproduction and audit procedure

Paper-only entry points are `paper/scripts/reduce_final_evidence.py`, `generate_final_tables.py`, `generate_final_figures.py`, and `verify_campaign_claims.py`. Historical raw checks are in `paper/audit/verify_historical_raw.py`. Reductions go to `paper/audit/reduced/`; their parsed CSV rows match retained originals. Android values are regenerated from retained aggregate records, not unavailable raw series. Numerical tables and figure inputs are traceable through `paper/audit/figure-sources.json`. A clean LaTeX build, citation/reference/box checks, extracted text, and page renders are recorded in the paper verification output. No command in this authoring workflow creates cloud resources or runs a performance benchmark.